[Nam Le] Linux User-Kernel Communication

1. Overview of User-Kernel Communication

Fast Compare

Feature	Syscal	IOCTL	Procfs	Netlink Socket
Nature	Standard API function of OS	The command **controlling device**	Virtual File System	Socket Mechanism (IPC)
Dimension	one-way (from User)	one-way (from User)	one-way (Read/write file)	2 ways
Data type	Function arguments	Command + Data Structure	Raw context	Packet / Byte stream
Purpose	Call a **service** is provided by Kernel	Send a controlling command to **device** or **kernel subsystem**	Read/write the information through `/proc` **file**	Exchange message between user/kernel

Background Description

1. Syscall: User call `read()`. Kernel perform read the data and return the result

- Pros: Safe,Available, the nature of LINUX
- Cons: Fixed and hard for extend

2. IOCTL: User call `ioctl(fd, command, arg)` to send a special command to driver through **file descriptor**

- When syscal functions dont have authority for control a specific hardware device, **IOCTL** do this (send to **driver**)

    - Pros: Allow dev-er define Driver on their own, dont need change source code of OS
    - Cons: Dont have standard, then meet **protect error** or crash all system (If not have the inspection in USER)

3. profs : User read/write on file in `/proc`. Kernel create the content or handle this work (not Regular File on Disk)
4. Netlink Socket: User send a message to Kernel and maybe receive a response or notification from kernel

2. System Call interface (Deep-Dive)

2.1 Basic Knowledge

As previous topic, we know that:

- User space: including **Applications**, **glibc**, **Shell**. These components only access limited memory is allocated for User Space, and can not access/control directly hardware
- Kernel Space: Kernel work on **Supervisor mode**(highest), can whole authority access memory and control directly **CPU, RAM, I/O device, Disk**

--> The question is why User cannot access to Kernel resource? **THERE ARE 4 REASONS**

1. The survival of whole System
   It means that seperating the authority, If an application in User have fault, drag on the crash on system. The applicaton must placed in User space
2. The fair in Multi-Tasking
3. Ping Topology from CPU
4. Memory Paging (belong to save mechanism)

The differ from syscall with normal C function

Feature	C function	Syscall
Space	User Space	Convert User to Kernel Space
Jump Mechanism	Normal jump command of CPU: `call` in x86	Hardware Interruption/Software Trap: `SYSCALL` or `INT 0x80`
Execution Cost	Very fast	Very Slow (Due to context switch)

Feature	C function	Syscall
Authority Access	Only RAM is provided to application	Hardware, I/O, Disk, RAM
Destinaton Addr	Know the address (Linking)	Dont know the address in kernel, only transmit **Syscall ID**

2.2 **CONTEXT SWITCH** mechanism:

1. Application process call a wrapper function in GLIBC library
2. This wrapper function copy the arguents to CPU register and push **system call number** to `eax` register
3. This wrapper function execute a **hardware trap instruction** like `int 0x80` or `sysenter` on x86 --> convert User Mode to Kernel Mode
4. `system_call` of Kernel search `sys_call_table` by corresponding executation internal number
5. Completely, kernel return the result to register, refresh State and convert CPU to User mode

This picture will describe the CONTEXT SWITCH mechanism

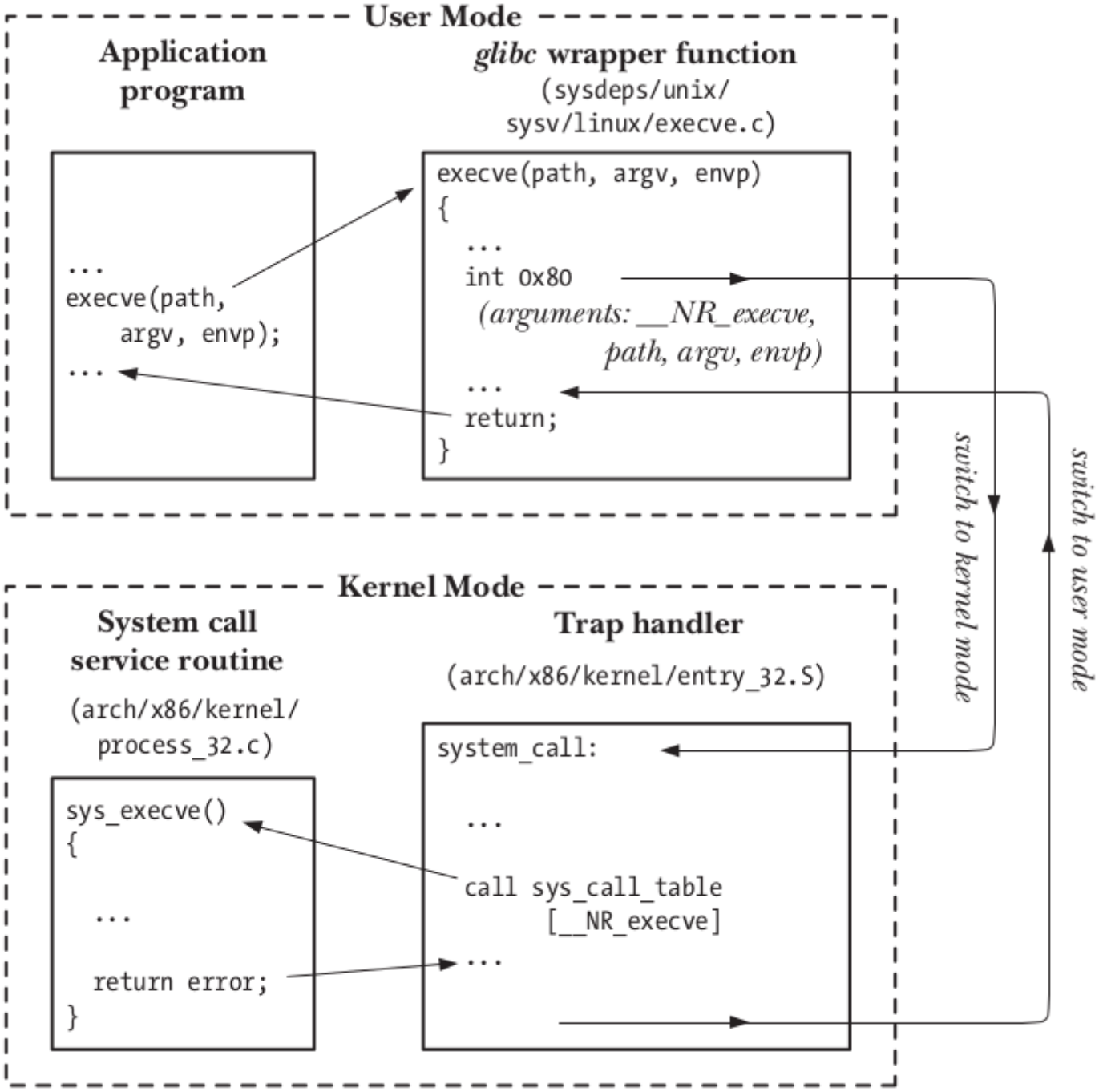


Figure 3-1: Steps in the execution of a system call

Example: The different between `getpid()` vs `syscall(SYS_getpid)`

```
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>
int main() {
    pid_t pid_normal = getpid();
    printf("Normal function getpid(): %d\n", pid_normal);
```

```
    pid_t pid_syscall = syscall(SYS_getpid);
    printf("Direct System Call: %d\n", pid_syscall);
    return 0;
}
```

Run: `strace ./my_program` Result:

```
getpid()                                = 18492
newfstatat(1, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x1), ...}, AT_EMPTY_PATH) = 0
getrandom("\x97\xe4\x36\x40\xe0\xd1\x97\x1d", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x5c31b9a86000
brk(0x5c31b9aa7000)                     = 0x5c31b9aa7000
write(1, "PID of this process = 18492\n", 28PID of this process = 18492
) = 28
-------------------------------------------------------------------------------------------
-
getpid()                                = 18492
write(1, "[Syscall] PID of this process = "..., 38[Syscall] PID of this process = 18492
) = 38
```

1. Half of top: `getpid()` **inside function**

- There will be occurs **Context Switch** first -> CPU jump Ring 3 to Ring 0 to ask Kernel
- Kernel return 18492, then CPU increase authority to Ring 3

2. Half of bottom: `syscall(SYS_getpid)`

- Skip all inspection step **of GLIBC**
- CPU load number 39 to RAX register and active `SYSCALL` command to move rightly to Ring 0

3. IOCTL interface (Input/Output Control)

In overview section, I said that **syscall** not enough for perform any change on **device driver** --> IOCTL born to do this
NOTE THAT , `ioctl()` is also a system call

Prototype:

```
#include <sys/ioctl.h>
int ioctl(int fd, int request, ... /* argp */);
                                /* Value returned on success depends on request, or –1 on
error */
```

- `fd`: file descriptor of deivce or file
- `request`: specified a number code for controlling device
- `argp`: is a pointer to either an integer or a structure; in some cases, it is unsed (use for exchange 2-ways data between User vs Kernel)

EXAMPLE Turn on the LED of CAPLOCK:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <sys/kd.h>     /* for KDSETLED and LED constants */

int main() {
    int fd = open("/dev/tty0", O_WRONLY);
    if (fd < 0) {
        perror("Error open file. Try with sudo");
        exit(1);
    }

    printf("===Sending IOCTL to keyboard===\n");

    printf("Turn on the led of Caps Lock...\n");
    if (ioctl(fd, KDSKBLED, LED_CAP) < 0) {
        perror("Error when turn on LED");
        close(fd);
        exit(1);
    }

    sleep(5);

    printf("Turn off Led\n");
    ioctl(fd, KDSETLED, 0xFF); // 0xFF notify to kernel that return the control for OS

    close(fd);
```

```
    return 0;
}
```

Run: `sudo ./caplock_led`
Result: Caplock led will be Light in 5s

Before explain the work's way of `ioctl` function. I will verify that **Although the led is turn on, But when we write, It's only normal character (not Upper Character)**:

- When we choose the Caplock on keyboard: (2 tasks in parallel)
  - Logic handle: Kernel turn on **flag** `Shift Lock` --> any character is entered from keyboard is upper
  - Physical handle: Kernel send a command to Keyboard through communication port to turn on the Led
- And the `ioctl()` only do the second step

We can change ioctl function to `ioctl(fd, KDSKBLED, KBD_CAPSLOCK);` --> do both 2 tasks

What occurs inside `ioctl` turn on the led function?
There will be go in 4 architecture layer of Kernel

1. VFS

- see fd and find the corresponding `struct file`
- IF fd is valid --> check **navigation panel** of `.unlocked_ioctl` of Driver which manage this file

2. TTY Subsystem & Input Driver

- Handle the code command: TTY Driver receive **KDSETLED** --> switch-case
- Check the authority: no `sudo`, no permission
- TTD driver call the lower layer **Input SubSystem**. Kernel configure Physical State of LED

3. Hardware-Specific Driver - Communicate with IC

- Driver USB will packeted a control packet. Send to Hardware

4. Hardware

- Microcontroller of keyboard read the packet, permit Electric Current pass and turn on the led of CAPLOCK

4. procfs (`/proc` Virtual File System)

4.1 Fast Overview

Nature: `proc` is a Virtual File System. It means, the files and the subdirectories that it contains **dont** reside on a disk. Instead, Kernel creates *immediately* (**on the fly**) in RAM for each time processes access
Problem: In the older UNIX implementations, it's not easy way to introspectively analyze (or change) the attributes of the Kernel, we must **verify** some question:

- How many processes are running on the system and who owns them?
- What files does a process have open?
- What files are currently locked, and which processes hold the locks?
- What socket are being used on the system?

--> Some older UNIX implementations solved this problem by allow privileged programs to **delve** into data structures in **kernel memory**. But it's suffered various problems. **In particular**, it required **specialized knowledge** of the kernel **data structures**, and these structures might change from one kernel **version** to the next, then requiring programs that **depend on** them to be written

Solution: provide easier access to kernel information, it's `/proc` vfs
It means **expose** *data structures of Kernel and the attribute of processes* to **User Space** under I/O file system.

To Operate:

- I/O file: read(), write(), open()
- shell: `cat`, `echo`

4.2 Main Structures

4.2.1 `/proc/PID` directory

Obtaining Information About a Process:

- `/proc/PID/fd`: directory containing symbolic links to file opened by this process
- `/proc/PID/maps`: memory mappings
- `/proc/PID/status`: Various Information

  .....

Example: The information about the `systemd` process, has process ID 1
Check the status of it `sudo cat /proc/1/status`

```
Name:	systemd
Umask:	0000
State:	S (sleeping)
Tgid:	1                                       /* Thread group ID (traditional PID,
getpid()) */
Ngid:	0
Pid:	1
PPid:	0
TracerPid:	0                                   /* PID of tracing process (0 if not traced)
*/
Uid:	0	0	0	0                       /* Real, effective, saved set, and FS */
Gid:	0	0	0	0
FDSize:	256                                     /*  of file descriptor slots currently
allocated */
Groups:	
NStgid:	1
NSpid:	1
NSpgid:	1
NSsid:	1
Kthread:	0
VmPeak:	  232120 kB
VmSize:	  166932 kB
VmLck:	       0 kB
VmPin:	       0 kB
VmHWM:	   12160 kB
VmRSS:	   12160 kB
RssAnon:	    3784 kB
RssFile:	    8376 kB
RssShmem:	       0 kB
VmData:	   20044 kB
VmStk:	     132 kB
VmExe:	     896 kB
VmLib:	    9064 kB
```

```
VmPTE:      92 kB
VmSwap:       0 kB
HugetlbPages:       0 kB
CoreDumping:0
THP_enabled:1
untag_mask:0xffffffffffffffff
Threads:1
SigQ:1/61781
SigPnd:0000000000000000
ShdPnd:0000000000000000
SigBlk:7be3c0fe28014a03
SigIgn:0000000000001000
SigCgt:00000001000004ec
CapInh:0000000000000000
CapPrm:000001fffffffff
CapEff:000001fffffffff
CapBnd:000001fffffffff
CapAmb:0000000000000000
NoNewPrivs:0
Seccomp:0
Seccomp_filters:0
Speculation_Store_Bypass:thread vulnerable
SpeculationIndirectBranch:conditional enabled
Cpus_allowed:fff
Cpus_allowed_list:0-11
Mems_allowed:00000000,00000000,00000000,00000000,
00000000,00000000,00000000,00000000,00000000,00000000,
00000000,00000000,00000000,00000000,00000000,00000000,
00000000,00000000,00000000,00000000,00000000,00000000,
00000000,00000000,00000000,00000000,00000000,00000000
,00000000,00000000,00000000,00000001
Mems_allowed_list:0
voluntary_ctxt_switches:7966
nonvoluntary_ctxt_switches:427
x86_Thread_features:
x86_Thread_features_locked:
```

Example: Check the memory maps of a program `sudo cat /proc/8554/maps`

```
5b3619af9000-5b3619afa000 r--p 00000000 103:06 2492195
/home/.../User_Kernel/caplock_led
5b3619afa000-5b3619afb000 r-xp 00001000 103:06 2492195
/home/.../User_Kernel/caplock_led
5b3619afb000-5b3619afc000 r--p 00002000 103:06 2492195
/home/.../User_Kernel/caplock_led
5b3619afc000-5b3619afd000 r--p 00002000 103:06 2492195
/home/.../User_Kernel/caplock_led
5b3619afd000-5b3619afe000 rw-p 00003000 103:06 2492195
/home/.../User_Kernel/caplock_led
5b36416dc000-5b36416fd000 rw-p 00000000 00:00 0                          [heap]
750e53800000-750e53828000 r--p 00000000 103:05 132762                    /usr/lib/x86_64-
linux-gnu/libc.so.6
750e53828000-750e539bd000 r-xp 00028000 103:05 132762                    /usr/lib/x86_64-
linux-gnu/libc.so.6
750e539bd000-750e53a15000 r--p 001bd000 103:05 132762                    /usr/lib/x86_64-
linux-gnu/libc.so.6
750e53a15000-750e53a16000 ---p 00215000 103:05 132762                    /usr/lib/x86_64-
linux-gnu/libc.so.6
750e53a16000-750e53a1a000 r--p 00215000 103:05 132762                    /usr/lib/x86_64-
linux-gnu/libc.so.6
750e53a1a000-750e53a1c000 rw-p 00219000 103:05 132762                    /usr/lib/x86_64-
linux-gnu/libc.so.6
750e53a1c000-750e53a29000 rw-p 00000000 00:00 0
750e53aab000-750e53aae000 rw-p 00000000 00:00 0
750e53abe000-750e53ac0000 rw-p 00000000 00:00 0
750e53ac0000-750e53ac2000 r--p 00000000 103:05 132751                    /usr/lib/.../ld-
linux-x86-64.so.2
750e53ac2000-750e53aec000 r-xp 00002000 103:05 132751                    /usr/lib/.../ld-
linux-x86-64.so.2
750e53aec000-750e53af7000 r--p 0002c000 103:05 132751                    /usr/lib/.../ld-
linux-x86-64.so.2
750e53af8000-750e53afa000 r--p 00037000 103:05 132751                    /usr/lib/.../ld-
linux-x86-64.so.2
750e53afa000-750e53afc000 rw-p 00039000 103:05 132751                    /usr/lib/.../ld-
linux-x86-64.so.2
7ffcfbf64000-7ffcfbf85000 rw-p 00000000 00:00 0                          [stack]
7ffcfbfe0000-7ffcfbfe4000 r--p 00000000 00:00 0                          [vvar]
7ffcfbfe4000-7ffcfbfe6000 r-xp 00000000 00:00 0                          [vdso]
```

```
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0                  [vsyscall]
```

4.2.2 `/proc/sys/` Directory

Allow view and modify the configure information of Kernel runtime (the PID max or the limit of file system buffer) Example: View and modify the PID max of kernel
The PID max default: `cat /proc/sys/kernel/pid_max`

```
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ cat /proc/sys/kernel/pid_max
4194304
```

Change the PID max value:

```
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ sudo echo 2000000 > /proc/sys/kernel/pid_max
bash: /proc/sys/kernel/pid_max: Permission denied
```

Permission denied because `echo` run under **User** authority before pushed to **sudo** .
Bash Shell in User Space see the `>` first
So, Shell open `/proc/sys/kernel/pid_max` first to overwrite
Change command:

```
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ echo 2000000 | sudo tee
/proc/sys/kernel/pid_max
2000000
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ cat /proc/sys/kernel/pid_max
2000000
```

`tee` function. We can understand tee is (T-connector, like T is a water pipe)
Left and Right section is run in parallel
`tee` will read the data from Left of pipe, print this data out of display
`tee` also open `/proc/sys/kernel/pid_max` to write into it

NOTE THAT: It's temporary change, will be reset to default if restart

Some **restrictions** apply when accessing these files:

- Some `/proc` files are read-only, only display it. (Most files under `/proc/PID`)
- Some `/proc` files can be read only by the file owner (or by a priviledged process). Example, all files `/proc/PID` are owner by the user who owns the corresponding process, some of these files (ex `/proc/PID/environ`), read permission is granted only to the file owner
- Other than files `/proc/PID` subdirectories, most files under /proc are owned by `root`, the files that are modifiable can be modified only by `root`

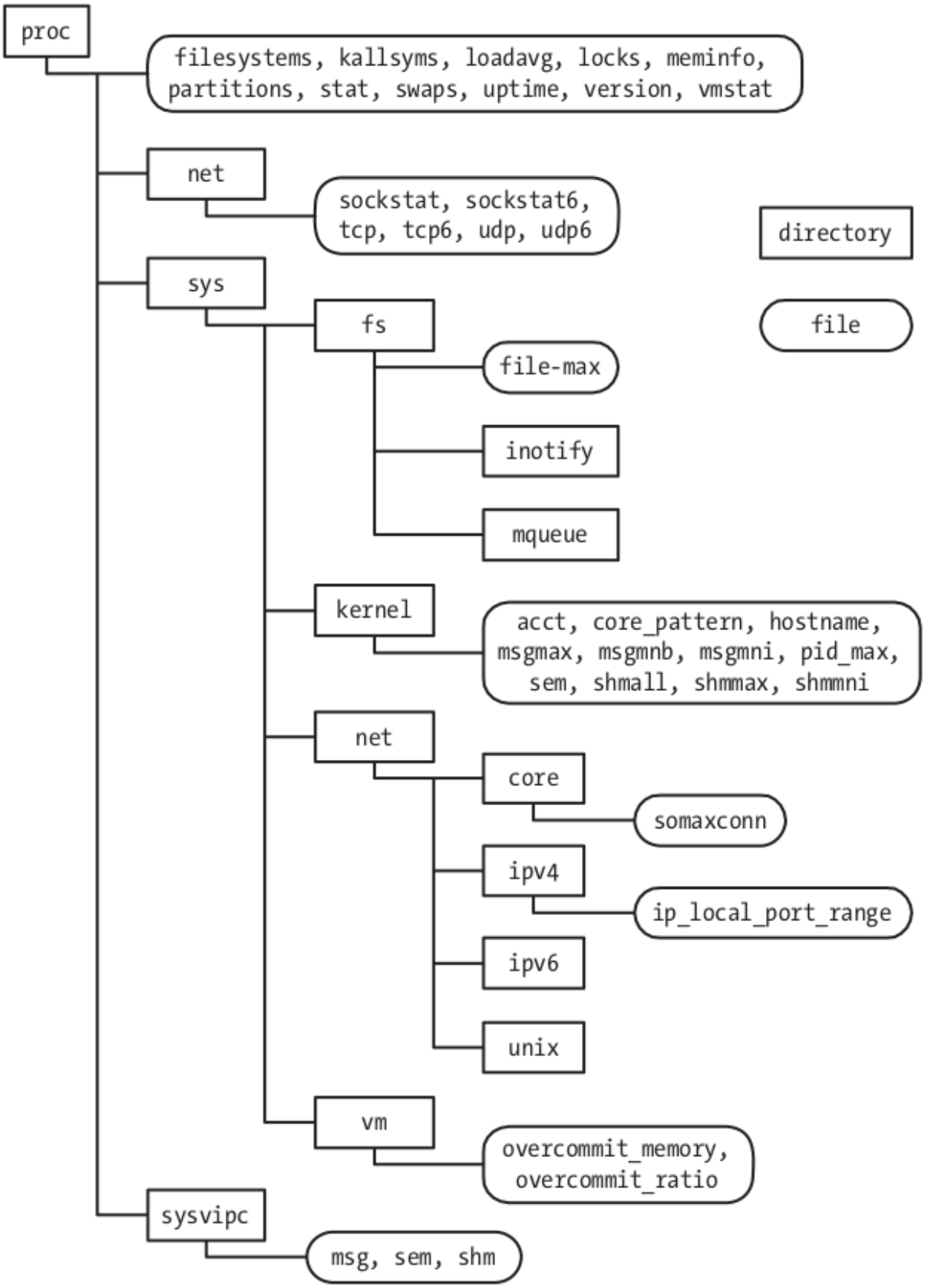


Figure 12-1: Selected files and subdirectories under /proc

5. Netlink Socket

5.1 The advantages of Socket (then Short Overview about Netlink Socket)

We will review the section Socket, there are some advantages about Socket:

- Bidirectional simultaneous data transfer
- Lossless on the internet, with TCP on transport layer. And of course on the localhost.
- High-speed data transfer, especially on localhost
- Flow control
- Asynchronous communication, message can be queued, sender does not have to wait for the receiver (not Full-Queue)
- Compare **Normal Socket** vs **Netlink Socket**
  - With communication in **User vs Kernel**. Other than previous methods, **user space** app must **initiate** the transfer to the kernel space; with **netlink sockets** *the kernel can initiate a tranfer*
  - Also, the previous mechanisms with **the various interface files** being strewn all over the **filesystem** can cause **kernel namespace pollution**. But with **netlink sockets**, this isn't the case as there are no files
- **Netlink Socket** support both **Unicast** and **Broadcast**:
  - to a specific PID (unicast)
  - Transmit a packet to a Group Process in User Space (broadcast)

5.2 Identify Mechanism and Netlink development

Netlink Socket is identified through PID (**Port ID**):

- UNIX Domain Socket use file descriptor
- Network socket use IP address
- Netlink socket use **PID (Port ID)** to identify communication space. The address of Kernel is initialized with PID = `0`

To deveplop a netlink socket and understand it, we need 2 programs (at a minimum) - one as the user and one as the kernel. User will send a message to **kernel module**, kernel module receive it and and print it. Then reply to the user space process, which is blocking on this

Example:

```
/* user_app.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <linux/netlink.h> /* Netlink structure */

#define NETLINK_USER 31
#define MAX_PAYLOAD 1024

int main() {
    int sock_fd = socket(AF_NETLINK, SOCK_RAW, NETLINK_USER);
    if (sock_fd < 0) {
        perror("socket");
        return 1;
    }
    struct sockaddr_nl src_addr, dest_addr;
    struct nlmsghdr *nlh = NULL;
    struct iovec iov;
    struct msghdr msg;

    /* Source address */
    memset(&src_addr, 0, sizeof(src_addr));
    src_addr.nl_family = AF_NETLINK;
    src_addr.nl_pid = getpid();      /* not Process ID, actually port id*/
    if (bind(sock_fd, (struct sockaddr *)&src_addr, sizeof(src_addr)) < 0) {
        perror("bind");
        return 1;
    }

    /* Destination address */
    memset(&dest_addr, 0, sizeof(dest_addr));
    dest_addr.nl_family = AF_NETLINK;
    dest_addr.nl_pid = 0;    /* for kernel */
```

```
    /* Pack the payload "Hello Kernel" */
    nlh = (struct nlmsghdr *)malloc(NLMSG_SPACE(MAX_PAYLOAD));
    memset(nlh, 0, NLMSG_SPACE(MAX_PAYLOAD));
    nlh->nlmsg_len = NLMSG_SPACE(MAX_PAYLOAD);
    nlh->nlmsg_pid = getpid();
    nlh->nlmsg_flags = 0;
    strcpy(NLMSG_DATA(nlh), "Hello Kernel!");

    /* Configure I/O vector to send/receive through Socket API*/
    iov.iov_base = (void *)nlh;
    iov.iov_len = nlh->nlmsg_len;

    /* Set up the message header structure */
    memset(&msg, 0, sizeof(msg));
    msg.msg_name = (void *)&dest_addr;
    msg.msg_namelen = sizeof(dest_addr);
    msg.msg_iov = &iov;
    msg.msg_iovlen = 1;  /* elements in msg_iov */

    /* Send message */
    printf("Send message to Kernel...\n");
    int res = sendmsg(sock_fd, &msg, 0);
    if (res < 0) {
        perror("L□i sendmsg");
    } else {
        printf("Đã g□i thành công %d bytes sang Kernel!\n", res);
    }

    close(sock_fd);
    free(nlh);
    return 0;
}
```

```
/* kernel_app.c */
#include <linux/module.h>
#include <linux/netlink.h>
#include <linux/skbuff.h>
#include <net/sock.h>
```

```
MODULE_LICENSE("GPL");
#define NETLINK_USER 31

struct sock *nl_sk = NULL;

/* Callback function. Kernel self-active whenever Netlink packet is sent to port 31*/
static void netlink_recv_and_reply(struct sk_buff *skb) {
    struct nlmsghdr *nlh;
    int pid;

    /* Unpack the packet, get the Header */
    nlh = (struct nlmsghdr *)skb->data;
    printk(KERN_INFO "Kernel received: %s\n", (char *)NLMSG_DATA(nlh));

    /*get pid of user space process has just sent (to reply)*/
    pid = nlh->nlmsg_pid;
}
static int __init hello_init(void) {
    struct netlink_kernel_cfg nl_kernel_cfg = {
        .input = netlink_recv_and_reply,
    };

    /* Create Netlink Socket inside Kernel */
    nl_sk = netlink_kernel_create(&init_net, NETLINK_USER, &nl_kernel_cfg);
    if (!nl_sk) {
        printk(KERN_ALERT "Error create Netlink Socket in Kernel.\n");
        return -ENOMEM;
    }
    printk(KERN_INFO "Kernel Netlink Module is available!\n");
    return 0;
}
static void __exit hello_exit(void) {
    netlink_kernel_release(nl_sk);
    printk(KERN_INFO "Kernel Netlink Module is deleted\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

To run:
Kernel

```
make
sudo insmod kernel_app.ko
```

User: `./user_app`
Then, in Kernel, `sudo dmesg | tail -n 10` to see 10 last lines of kernel notfication

```
[30743.706826] audit: type=1400 audit(1792405166.688:431): apparmor="ALLOWED"
operation="uring_sqpoll" class="io_uring" profile="snap.code.code" pid=25923 comm="cpptools-
srv2" requested="sqpoll" denied="sqpoll"
[31179.114431] audit: type=1400 audit(1792405602.112:432): apparmor="ALLOWED"
operation="uring_sqpoll" class="io_uring" profile="snap.code.code" pid=26168 comm="cpptools-
srv" requested="sqpoll" denied="sqpoll"
[32428.056108] Kernel Netlink Module is deleted
[32448.945612] Kernel Netlink Module is available!
[32585.443729] audit: type=1400 audit(1792407008.441:433): apparmor="ALLOWED"
operation="uring_sqpoll" class="io_uring" profile="snap.code.code" pid=27574 comm="cpptools-
srv2" requested="sqpoll" denied="sqpoll"
[32636.705232] Kernel Netlink Module is deleted
[32648.323468] Kernel Netlink Module is available!
[32687.531402] audit: type=1400 audit(1792407110.524:434): apparmor="ALLOWED"
operation="uring_sqpoll" class="io_uring" profile="snap.code.code" pid=28079 comm="cpptools-
srv2" requested="sqpoll" denied="sqpoll"
[32697.343765] audit: type=1400 audit(1792407120.337:435): apparmor="ALLOWED"
operation="uring_sqpoll" class="io_uring" profile="snap.code.code" pid=28096 comm="cpptools-
srv" requested="sqpoll" denied="sqpoll"
[32845.110684] Kernel received: Hello Kernel!
```

We can see Kernel received message in the last line!

Revision #7
Created 2026-10-15 02:44:34 UTC by Lê Hoài Nam
Updated 2026-10-19 10:58:35 UTC by Lê Hoài Nam